
Лабораторная работа №2

MariaDB: движки хранения, транзакции и полнотекстовый поиск

Дисциплина: Информационные системы и базы данных

Платформа: Windows 11 · Docker · Python 3.10+ · PyMySQL

Время выполнения: 4–6 академических часов

Студент _____ Группа _____

Дата выполнения _____

Цель работы

Исследовать три ключевых механизма MariaDB, отличающих её от легковесных СУБД:

- › Системы хранения (Storage Engines) — сравнить производительность и надёжность InnoDB, Aria и MyISAM при массовой вставке и имитации сбоя.
- › Уровни изоляции транзакций — наблюдать фантомные чтения при READ COMMITTED и REPEATABLE READ.
- › Полнотекстовый поиск — оценить скорость FULLTEXT-индекса в сравнении с оператором LIKE.

Подготовительный шаг

Перед выполнением задач запустите MariaDB и установите Python-зависимости.

1. Запуск MariaDB в Docker

Выполните в PowerShell:

```
docker run --name mariadb-lab \  
  -e MYSQL_ROOT_PASSWORD=root \  
  -p 3306:3306 \  
  -d mariadb:10.11  
  
# Проверьте, что контейнер запущен:  
docker ps
```

2. Установка Python-библиотеки

Выполните в PowerShell:

```
pip install pymysql
```

3. Заготовка скрипта lab2_mariadb.py

Создайте файл lab2_mariadb.py в VS Code или любом редакторе:

```

import pymysql
import time
import random
import threading
from contextlib import contextmanager

DB_CONFIG = {
    "host": "127.0.0.1",
    "user": "root",
    "password": "root",
    "database": "test",
    "autocommit": False
}

@contextmanager
def get_connection():
    """Контекстный менеджер подключения к БД."""
    conn = pymysql.connect(**DB_CONFIG)
    try:
        yield conn
    finally:
        conn.close()

def measure_time(func):
    """Декоратор: замеряет время через time.perf_counter()."""
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        return result, time.perf_counter() - start
    return wrapper

```

01

Сравнение Storage Engines: скорость вставки

InnoDB vs Aria vs MyISAM — кто быстрее при массовой вставке?

- ☐ Python — cursor.execute() для SQL
- ☐ Замер времени — time.perf_counter()
- ☐ Результат — скриншот терминала с таблицей чисел

1. Создайте три таблицы с разными движками:

```
def task1_storage_speed():
    with get_connection() as conn:
        with conn.cursor() as cur:
            for engine in ["InnoDB", "Aria", "MyISAM"]:
                table = f"test_{engine.lower()}"
                cur.execute(f"DROP TABLE IF EXISTS {table}")
                cur.execute(
                    f"CREATE TABLE {table} ("
                    f"  id INT AUTO_INCREMENT PRIMARY KEY,"
                    f"  val INT, txt VARCHAR(100)"
                    f") ENGINE={engine}"
                )
            conn.commit()
```

2. Вставьте 50 000 записей в каждую таблицу, повторите 3 раза:

```
results = {}
for engine in ["InnoDB", "Aria", "MyISAM"]:
    table = f"test_{engine.lower()}"
    times = []
    for attempt in range(3):
        cur.execute(f"TRUNCATE TABLE {table}")
        conn.commit()
        start = time.perf_counter()
        for i in range(50_000):
            cur.execute(
                f"INSERT INTO {table} (val, txt) VALUES (%s, %s)",
                (i, f"row_{i}")
            )
        conn.commit()
        times.append(time.perf_counter() - start)
    results[engine] = times

print(f"{'Engine':<10} {'Try1':<9} {'Try2':<9} {'Try3':<9} {'Avg':<9}")
print('-' * 48)
for engine, times in results.items():
    avg = sum(times) / 3
    t1, t2, t3 = times
    print(f"{'engine':<10} {'t1':<9.3f} {'t2':<9.3f} {'t3':<9.3f} {'avg':<9.3f}")
```

☐ Что сдаётся: скриншот терминала + заполненная таблица ниже

Движок	Попытка 1 (с)	Попытка 2 (с)	Попытка 3 (с)	Среднее (с)
InnoDB	...	...	...	...
Aria	...	...	...	...
MyISAM	...	...	...	...

02

Поведение при сбое: имитация аварийного отключения

Какой движок переживает перезапуск без потери незафиксированных данных?

- ☐ Python — основной терминал
- ☐ PowerShell — отдельное окно для docker restart
- ☐ Результат — скриншот с COUNT(*) после перезапуска

1. Создайте таблицы и вставьте 10 000 записей без COMMIT:

```
def task2_crash_test():
    with get_connection() as conn:
        with conn.cursor() as cur:
            for engine in ["InnoDB", "Aria", "MyISAM"]:
                table = f"crash_{engine.lower()}"
                cur.execute(f"DROP TABLE IF EXISTS {table}")
                cur.execute(
                    f"CREATE TABLE {table} ("
                    f"  id INT PRIMARY KEY, data VARCHAR(100)"
                    f") ENGINE={engine}"
                )
            conn.commit()

            for engine in ["InnoDB", "Aria", "MyISAM"]:
                table = f"crash_{engine.lower()}"
                for start in range(0, 10_000, 1_000):
                    batch = [(i, f"data_{i}") for i in range(start, start+1_000)]
                    cur.executemany(
                        f"INSERT INTO {table} (id, data) VALUES (%s, %s)", batch
                    )
            # COMMIT намеренно НЕ вызывается

            print("В ДРУГОМ терминале выполните: docker restart mariadb-lab")
            input("Нажмите Enter после перезапуска контейнера...")
```

2. После перезапуска подождите 10 секунд и проверьте данные:

```
time.sleep(10)
with get_connection() as conn2:
    with conn2.cursor() as cur2:
        for engine in ["InnoDB", "Aria", "MyISAM"]:
            table = f"crash_{engine.lower()}"
            cur2.execute(f"SELECT COUNT(*) FROM {table}")
            print(f"{table:<22}: {cur2.fetchone()[0]} записей")
```

☐ Что сдаётся: скриншот COUNT(*) + заполненная таблица + вывод 2–3 предложения

Таблица	Движок	Записей после сбоя	Ожидаемый результат
crash_innodb	InnoDB	...	0 (откат без COMMIT)
crash_aria	Aria	...	0 или частично
crash_myisam	MyISAM	...	10 000 (нет транзакций)

03

Уровни изоляции: фантомные чтения

READ COMMITTED vs REPEATABLE READ — что видит незавершённая транзакция?

- ☐ Python — два соединения через threading
- ☐ Результат — два скриншота терминала (RC и RR)

1. Подготовьте таблицу и начальные данные:

```
def task3_isolation_levels():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("DROP TABLE IF EXISTS isolation_test")
            cur.execute(
                "CREATE TABLE isolation_test "
                "(id INT PRIMARY KEY, val INT) ENGINE=InnoDB"
            )
            cur.executemany(
                "INSERT INTO isolation_test VALUES (%s, %s)",
                [(1, 10), (2, 20)]
            )
            conn.commit()
```

2. Сценарий А — READ COMMITTED (два потока одновременно):

```
def session1_rc():
    conn = pymysql.connect(**DB_CONFIG)
    cur = conn.cursor()
    cur.execute("SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED")
    cur.execute("START TRANSACTION")
    cur.execute("SELECT * FROM isolation_test")
    print(f"[RC] До вставки: {cur.fetchall()}")
    input("Нажмите Enter, когда сессия 2 выполнит вставку...")
    cur.execute("SELECT * FROM isolation_test")
    print(f"[RC] После вставки: {cur.fetchall()}")
    conn.commit(); conn.close()

def session2_insert():
    time.sleep(0.5)
    conn = pymysql.connect(**DB_CONFIG)
    cur = conn.cursor()
    cur.execute("INSERT INTO isolation_test VALUES (3, 30)")
    conn.commit()
    print("[RC] Сессия 2: строка (3, 30) вставлена")
    conn.close()

t1 = threading.Thread(target=session1_rc)
t2 = threading.Thread(target=session2_insert)
t1.start(); t2.start()
t1.join(); t2.join()
```

3. Сценарий Б — REPEATABLE READ: повторите аналогично, заменив уровень изоляции и очистив таблицу перед запуском.

□ **Что сдаётся: два скриншота (RC и RR) + сравнительная таблица**

Уровень изоляции	Строк до вставки	Строк после вставки	Фантом?
READ COMMITTED	2	3	Да — новая строка видна
REPEATABLE READ	2	2	Нет — снимок зафиксирован

04

Полнотекстовый поиск: LIKE vs MATCH() AGAINST()

Насколько FULLTEXT-индекс быстрее полного сканирования таблицы?

- Python — генерация данных и замеры
- Результат — скриншот с временами + вывод EXPLAIN

1. Создайте таблицу и вставьте 100 000 записей:


```

WORDS = [
    "apple", "banana", "orange", "grape", "cherry", "mango",
    "peach", "plum", "strawberry", "blueberry", "raspberry",
    "blackberry", "watermelon", "melon", "pear", "kiwi",
    "pineapple", "lemon", "lime", "coconut"
]

def generate_text(idx):
    selected = random.sample(WORDS, 12)
    if idx % 10 == 0:
        selected.append("apple") # ~10% записей содержат "apple"
    random.shuffle(selected)
    return " ".join(selected)

def task4_fulltext_speed():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("DROP TABLE IF EXISTS articles")
            cur.execute(
                "CREATE TABLE articles ("
                " id INT AUTO_INCREMENT PRIMARY KEY,"
                " title VARCHAR(200), body TEXT"
                ") ENGINE=InnoDB"
            )
            conn.commit()
            for start in range(0, 100_000, 1_000):
                batch = [(f"title_{i}", generate_text(i))
                        for i in range(start, start+1_000)]
                cur.executemany(
                    "INSERT INTO articles (title, body) VALUES (%s,%s)", batch
                )
                conn.commit()

```

2. Замер А — LIKE без индекса. Затем создайте FULLTEXT и выполните замер Б:

```

# Замер A: LIKE
t0 = time.perf_counter()
cur.execute("SELECT COUNT(*) FROM articles WHERE body LIKE '%apple%'")
count_like = cur.fetchone()[0]
time_like = time.perf_counter() - t0
print(f"LIKE: {time_like:.4f} с найдено: {count_like}")

# Создаём FULLTEXT-индекс
cur.execute("ALTER TABLE articles ADD FULLTEXT INDEX ft_idx (body)")
conn.commit()

# Замер Б: MATCH AGAINST
t0 = time.perf_counter()
cur.execute(
    "SELECT COUNT(*) FROM articles WHERE MATCH(body) AGAINST ('apple')"
)
count_ft = cur.fetchone()[0]
time_ft = time.perf_counter() - t0
print(f"MATCH AGAINST: {time_ft:.4f} с найдено: {count_ft}")

# EXPLAIN — скопируйте вывод в отчёт
cur.execute(
    "EXPLAIN SELECT COUNT(*) FROM articles WHERE body LIKE '%apple%'")
print("EXPLAIN LIKE: ", cur.fetchall())
cur.execute(
    "EXPLAIN SELECT COUNT(*) FROM articles"
    " WHERE MATCH(body) AGAINST('apple')"
)
print("EXPLAIN MATCH:", cur.fetchall())

```

□ Что сдаётся: скриншот с двумя временами и выводом EXPLAIN + заполненная таблица

Метод	Время (с)	Найдено строк	Использует индекс?
LIKE	X.XXXX	...	Нет — full table scan
MATCH AGAINST	Y.YYYY	...	Да — FULLTEXT index

05

Ранжирование результатов полнотекстового поиска

Natural Language Mode vs Boolean Mode — как работает релевантность?

- ☐ Python — SQL-запросы, вывод в терминал
- ☐ Результат — два скриншота с топ-10 результатов

Выполните два запроса — Natural Mode и Boolean Mode:

```
def task5_ranking():
    with get_connection() as conn:
        with conn.cursor() as cur:

            # Natural Language Mode
            cur.execute(
                "SELECT id, title, "
                "MATCH(body) AGAINST('apple orange') AS relevance "
                "FROM articles "
                "WHERE MATCH(body) AGAINST('apple orange') "
                "ORDER BY relevance DESC LIMIT 10"
            )
            print("\n=== Natural Language Mode ===")
            for row in cur.fetchall():
                print(f" ID:{row[0]:<7} {row[1]:<22} rel={row[2]:.4f}")

            # Boolean Mode: apple обязателен, banana запрещён
            cur.execute(
                "SELECT id, title, "
                "MATCH(body) AGAINST('+apple -banana' IN BOOLEAN MODE) AS relevance "
                "FROM articles "
                "WHERE MATCH(body) AGAINST('+apple -banana' IN BOOLEAN MODE) "
                "ORDER BY relevance DESC LIMIT 10"
            )
            print("\n=== Boolean Mode (+apple -banana) ===")
            for row in cur.fetchall():
                print(f" ID:{row[0]:<7} {row[1]:<22} rel={row[2]:.4f}")
```

- ☐ Что сдаётся: два скриншота + пояснение (3–4 предложения) про TF-IDF и разницу режимов

06

Влияние FULLTEXT-индекса на скорость вставки

Насколько медленнее вставка при наличии полнотекстового индекса?

- ☐ Python — весь код
- ☐ Результат — скриншот с двумя временами и % разницей

Измерьте время вставки 20 000 записей в таблицу без индекса и с FULLTEXT:

```
WORDS2 = ["database", "sql", "index", "query", "transaction",
          "lock", "deadlock", "isolation", "commit", "rollback",
          "backup", "recovery", "replication", "cluster"]

def generate_text2(i):
    return " ".join(random.sample(WORDS2, 10)) + f" {i}"

def task6_insert_overhead():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("DROP TABLE IF EXISTS articles_no_ft")
            cur.execute(
                "CREATE TABLE articles_no_ft "
                "(id INT AUTO_INCREMENT PRIMARY KEY, body TEXT) ENGINE=InnoDB"
            )
            cur.execute("DROP TABLE IF EXISTS articles_ft")
            cur.execute(
                "CREATE TABLE articles_ft ("
                " id INT AUTO_INCREMENT PRIMARY KEY,"
                " body TEXT, FULLTEXT INDEX ft_idx (body)"
                ") ENGINE=InnoDB"
            )
            conn.commit()
            data = [(generate_text2(i),) for i in range(20_000)]

            # Вставка БЕЗ индекса
            t0 = time.perf_counter()
            for s in range(0, 20_000, 1_000):
```

```

cur.executemany(
    "INSERT INTO articles_no_ft (body) VALUES (%s)", data[s:s+1_000]
)
conn.commit()
time_no = time.perf_counter() - t0

# Вставка C FULLTEXT-индексом
t0 = time.perf_counter()
for s in range(0, 20_000, 1_000):
    cur.executemany(
        "INSERT INTO articles_ft (body) VALUES (%s)", data[s:s+1_000]
    )
    conn.commit()
time_ft = time.perf_counter() - t0

overhead = (time_ft - time_no) / time_no * 100
print(f"Без индекса: {time_no:.3f} с")
print(f"C FULLTEXT: {time_ft:.3f} с")
print(f"Накладные расходы: +{overhead:.1f}%")

```

☐ Что сдаётся: скриншот с тремя строками вывода + заполненная таблица

Таблица	Индекс	Время вставки 20К (с)	Накладные расходы (%)
articles_no_ft	Нет	X.XXX	—
articles_ft	FULLTEXT	Y.YYY	+Z.Z%

Формат отчёта

Отчёт оформляется в формате Word, PDF или Markdown:

1. Титульный лист

› Название работы, ФИО студента, группа, дата

2. Цель работы

› Копируется из задания

3. Ход выполнения — для каждой из 6 задач

- › Краткое описание выполненных действий
- › Ключевые фрагменты кода
- › Таблица с числовыми результатами (заполненная)
- › Скриншот терминала с выводом
- › Вывод по задаче — 1–2 предложения

4. Общий вывод

- › Какой движок хранения для каких задач подходит
- › Как уровень изоляции влияет на конкурентные запросы
- › Когда стоит использовать FULLTEXT-индекс

5. Приложение

- › Полный листинг файла lab2_mariadb.py

Требования к скриншотам

Файл	Содержимое
task1.png	Таблица с временами вставки для трёх движков
task2.png	Вывод COUNT(*) после имитации сбоя
task3a.png	READ COMMITTED — строки до и после вставки
task3b.png	REPEATABLE READ — строки до и после вставки
task4.png	Времена LIKE и MATCH AGAINST + вывод EXPLAIN
task5.png	Топ-10 для Natural и Boolean Mode
task6.png	Времена вставки с индексом и без + % разница

! На каждом скриншоте должен быть виден вывод Python-скрипта в терминале. Допускается использование DBeaver или HeidiSQL для SQL-запросов.

Шкала оценивания

Задача	Баллы	Что проверяется
1	2	Работа с движками, замер и усреднение времени
2	2	Понимание ACID, поведение при сбоях
3	2	Уровни изоляции транзакций, работа с threading
4	2	Полнотекстовый индекс, интерпретация EXPLAIN
5	1	Ранжирование и булевый режим поиска
6	1	Накладные расходы на обновление индекса
Итого	10	

Снижение баллов

- › -1 балл — отсутствует скриншот для задачи
- › -1 балл — нет числовых результатов (только «всё работает»)

- › -0.5 балла — вывод по задаче отсутствует или формальный
- › -0.5 балла — код содержит ошибки или не запускается

Дополнительное задание (+1–2 балла)

Реализовать полнотекстовый поиск по двум полям (title + body) и сравнить релевантность с поиском только по body. Показать, что совпадение в заголовке повышает оценку.

Важные замечания

- › Замер времени — только через `time.perf_counter()`.
- › Транзакции — в PyMySQL по умолчанию `autocommit=False`; явный `commit()` обязателен.
- › `executemany` — принимает список кортежей: `[(val1,), (val2,)]`.
- › Перезапуск Docker — ждите 10 секунд: `time.sleep(10)`.
- › Порт 3306 занят — смените маппинг: `-p 3307:3306`.
- › Минимальная длина слова — MariaDB игнорирует слова короче 4 символов в `FULLTEXT`.